

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NGOẠI THƯƠNG



BÁO CÁO TIỂU LUẬN VÀ THỰC HÀNH
MÔN HỌC
MẠNG MÁY TÍNH
ĐỀ TÀI:
PHÂN TÍCH VÀ DỰ ĐOÁN THỜI GIAN PHẢN HỒI
CỦA DỊCH VỤ WEB TỪ CÁC ĐẶC TRƯNG MẠNG

Sinh viên thực hiện : **Đinh Khôi Nguyên**
Nguyễn Việt A
Nguyễn Cao Hoàng Minh
Nguyễn Sĩ Sơn
Dương Tiến An

Giảng viên : **TS Ngô Hải Anh**
Khoa : **Công nghệ và khoa học dữ liệu**
Lớp : **COSH201 (2526.1 – GD1).1**

Hà Nội – 12/9/2026

PHIẾU CHẤM ĐIỂM

Sinh viên thực hiện:

Họ và tên	Chữ ký	Ghi chú
Đinh Khôi Nguyên Nguyễn Việt A Nguyễn Cao Hoàng Minh Nguyễn Sĩ Sơn Dương Tiến An		

Giảng viên chấm:

Họ và tên	Điểm	Chữ ký
Ngô Hải Anh		

MỤC LỤC

LỜI MỞ ĐẦU.....	1
CHƯƠNG 1: TỔNG QUAN.....	3
1.1. Bối cảnh thực tế và vấn đề mạng cần nghiên cứu.....	3
1.2. Mục tiêu, câu hỏi nghiên cứu, phạm vi và phương pháp tổng quát	3
<i>1.2.1. Mục tiêu nghiên cứu</i>	<i>3</i>
<i>1.2.2. Câu hỏi nghiên cứu</i>	<i>4</i>
<i>1.2.3. Phạm vi nghiên cứu.....</i>	<i>4</i>
<i>1.2.4. Phương pháp tổng quát</i>	<i>4</i>
1.3. Kết quả dự kiến	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	6
2.1. Các giao thức nền tảng của một phiên truy cập Web.....	6
<i>2.1.1. DNS (Domain Name System).....</i>	<i>6</i>
<i>2.1.2. TLS và HTTPS.....</i>	<i>6</i>
<i>2.1.3. HTTP (Hypertext Transfer Protocol).....</i>	<i>6</i>
<i>2.1.4. TCP (Transmission Control Protocol).....</i>	<i>6</i>
<i>2.1.5. RTT (Round-Trip Time).....</i>	<i>7</i>
2.2. Throughput vs Goodput.....	7
2.3. Mất gói (Packet Loss) và cơ chế phản ứng của TCP	8
2.4. Jitter và cơ chế truy cập kênh Wi-Fi.....	8
2.5. Phân rã Response Time.....	9
CHƯƠNG 3: DATASET VÀ PHƯƠNG PHÁP	10
3.1. Nguồn dữ liệu và cách thu thập	10
<i>3.1.1. Nguồn dữ liệu.....</i>	<i>10</i>
<i>3.1.2 Quy trình thu thập dữ liệu</i>	<i>11</i>
3.2 Mô tả dữ liệu.....	12

3.3. Xử lý dữ liệu	13
3.3.1. <i>Tạo dataset.....</i>	<i>13</i>
3.3.2 <i>Xử lý dữ liệu.....</i>	<i>13</i>
3.3.3. <i>Tách Train/Test và làm sạch thống kê</i>	<i>14</i>
CHƯƠNG 4: XÂY DỰNG CHƯƠNG TRÌNH VÀ THỰC NGHIỆM.....	15
4.1. Kiến trúc chương trình	15
4.1.1. <i>Thu thập và nạp dữ liệu.....</i>	<i>15</i>
4.1.2. <i>Làm sạch dữ liệu</i>	<i>15</i>
4.1.3. <i>Phân tích khám phá dữ liệu.....</i>	<i>17</i>
4.1.4. <i>Trích xuất đặc trưng.....</i>	<i>18</i>
4.1.5. <i>Huấn luyện mô hình</i>	<i>19</i>
4.1.6. <i>Đánh giá</i>	<i>21</i>
4.1.7. <i>Trực quan hóa.....</i>	<i>22</i>
4.2. Cấu hình thực nghiệm và khả năng tái lập	23
4.3. Hướng dẫn chạy lại thực nghiệm	24
CHƯƠNG 5: KẾT QUẢ VÀ THẢO LUẬN.....	26
5.1. Đánh giá và so sánh các mô hình học máy	26
5.1.1 <i>Linear Regression</i>	<i>26</i>
5.1.2 <i>Decision Tree</i>	<i>28</i>
5.1.3 <i>Random Forest.....</i>	<i>30</i>
5.2. So sánh giữa các mô hình và với Baseline.....	31
5.3. Phân tích feature	32
5.4. Các trường hợp dự đoán sai	35
5.4.1. <i>Trường hợp model dự đoán thời gian nhanh hơn thực tế (Underprediction) .</i>	<i>35</i>

5.4.2. Trường hợp model dự đoán thời gian phản hồi chậm hơn thực tế (Overprediction)	36
5.5. Giới hạn của mô hình	37
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	38
6.1. Tóm tắt kết quả	38
6.2. Trả lời câu hỏi nghiên cứu	39
6.2.1. Các yếu tố đặc trưng mạng (DNS, TCP, TLS, RTT) ảnh hưởng đến tổng thời gian phản hồi ra sao?	39
6.2.2. Các yếu tố ngữ cảnh (thời điểm truy cập, phân loại domain, kích thước phản hồi) ảnh hưởng thế nào?	39
6.2.3. Mô hình tuyến tính và phi tuyến tính phù hợp ra sao? Có vượt Baseline không? Mô hình nào tốt nhất?	40
6.3. Hạn chế	40
6.3.1. Thiếu sót của mô hình	40
6.3.2. Hạn chế của mô hình.....	40
6.4. Hướng phát triển	41
TÀI LIỆU THAM KHẢO.....	42

DANH MỤC HÌNH ẢNH

STT	Hình ảnh/ Code/ Biểu đồ	Tên	Vị trí
1	Bảng 3.1.1	15 URL phổ biến dùng để tạo dataset	Mục 3.1.1
2	Bảng 3.2	Data dictionary	Mục 3.2
3	Bảng 3.3.3	Tách tập dữ liệu Train/Test	Mục 3.3.3
4	Đoạn mã 4.1.1	Lưu dữ liệu Dataset_finale.csv	Mục 4.1.1
5	Đoạn mã 4.1.2.1	Chia tập train/test	Mục 4.1.2
6	Đoạn mã 4.1.2.2	Xử lý outlier bằng Isolation Forest	Mục 4.1.2
7	Đoạn mã 4.1.5.2	Huấn luyện 4 mô hình	Mục 4.1.5
8	Bảng 4.1.5.1	Các mô hình và baseline được lựa chọn	Mục 4.1.5
9	Bảng 4.1.7	Tổng hợp các nhóm biểu đồ được sinh bởi chương trình	Mục 4.1.7
10	Bảng 4.2	Cấu hình Baseline và các mô hình	Mục 4.2
11	Hình 5.1.1.1	Actual vs Predicted — Linear Regression	Mục 5.1.1
12	Hình 5.1.1.2	Phân bố residual — Linear Regression	Mục 5.1.1
13	Hình 5.1.2.1	Actual – Predicted — Decision Tree	Mục 5.1.2
14	Hình 5.1.2.2	Phân bố residual — Decision Tree	Mục 5.1.2
15	Hình 5.1.3.1	Actual – Predicted — Random Forest	Mục 5.1.3
16	Hình 5.1.3.2	Phân bố residual — Random Forest	Mục 5.1.3
17	Bảng 5.2	Kết quả đánh giá trên tập test (172 bản ghi)	Mục 5.2
18	Hình 5.3	Tương quan Pearson giữa các đặc trưng và response_time_ms	Mục 5.3

LỜI MỞ ĐẦU

Trong bối cảnh các ứng dụng web ngày càng phổ biến, thời gian phản hồi của hệ thống là một yếu tố quan trọng ảnh hưởng trực tiếp đến trải nghiệm người dùng cũng như hiệu quả hoạt động của ứng dụng. Việc đo đạc và phân tích thời gian phản hồi giúp đánh giá khả năng đáp ứng của hệ thống trong các điều kiện mạng khác nhau, đồng thời cung cấp cơ sở để dự đoán và tối ưu hiệu năng.

Trong báo cáo này, nhóm tiến hành đo đạc thời gian phản hồi của một số yêu cầu web, từ đó phân tích dữ liệu thu được và xây dựng mô hình nhằm mô phỏng, dự đoán và ước lượng thời gian phản hồi của hệ thống. Bài báo cáo gồm có 6 chương, trong đó bao gồm:

Chương 1. Tổng quan về vấn đề nghiên cứu: chương này giới thiệu mục tiêu và câu hỏi nghiên cứu, đồng thời trình bày phạm vi nghiên cứu và phương pháp nghiên cứu tổng quan

Chương 2. Cơ sở lý thuyết: chương này trình bày các khái niệm cơ bản của mạng máy tính nói chung và các khái niệm đặc trưng về thời gian phản hồi của web nói riêng

Chương 3. Dataset và phương pháp: chương này đưa ra nguồn dữ liệu của báo cáo, cách thu thập cũng như phương pháp xử lý dữ liệu của nhóm

Chương 4. Xây dựng chương trình và thực nghiệm: chương này mô tả kiến trúc chương trình của mô hình nhóm xây dựng, đồng thời đưa ra quy trình huấn luyện mô hình cũng như các tiêu chí đánh giá cần có

Chương 5. Kết quả và thảo luận: chương này đưa ra, đánh giá và so sánh kết quả báo cáo của các mô hình cũng như chỉ ra các giới hạn và trường hợp dự đoán sai của chúng

Chương 6. Kết luận và hướng phát triển: chương này tóm tắt kết quả báo cáo, trả lời các câu hỏi nghiên cứu đồng thời đưa ra định hướng phát triển xa hơn cho mô hình và quy trình nghiên cứu trong tương lai

Qua đó, nhóm có thể áp dụng các kiến thức đã học trong môn Mạng máy tính vào một bài toán thực tế, đồng thời đánh giá mức độ phù hợp của mô hình với dữ liệu đo đạc. Kết quả của quá trình này giúp nhóm có cái nhìn rõ hơn về các yếu tố ảnh hưởng đến thời gian phản hồi của web, cũng như khả năng sử dụng dữ liệu thực tế và mô hình hóa để phục vụ việc phân tích, dự đoán hiệu năng mạng.

Sau đây là đường liên kết dẫn tới code, dataset hoàn chỉnh của nhóm: [Link](#)

CHƯƠNG 1: TỔNG QUAN

1.1. Bối cảnh thực tế và vấn đề mạng cần nghiên cứu

Trong hạ tầng Internet hiện đại, dịch vụ Web (HTTP/HTTPS) là nền tảng cốt lõi cho phần lớn các ứng dụng truyền thông, thương mại và giải trí. Theo báo cáo của Google, có tới 53% người dùng sẽ rời bỏ một trang Web nếu thời gian tải vượt quá 3 giây, cho thấy trải nghiệm người dùng phụ thuộc trực tiếp vào thời gian phản hồi của dịch vụ, kết quả tổng hòa của chuỗi độ trễ phát sinh qua từng giai đoạn trong chồng giao thức TCP/IP, từ phân giải tên miền, thiết lập kết nối vận chuyển, bắt tay bảo mật cho đến xử lý và truyền tải nội dung ở tầng ứng dụng.

Xuất phát từ thực tế đó, báo cáo này tập trung nghiên cứu, phân tích mối quan hệ định lượng giữa độ trễ các thành phần ở tầng giao vận, tầng ứng dụng, kích thước dữ liệu tải về và loại hình dịch vụ, với tổng thời gian phản hồi dịch vụ Web. Việc làm rõ tỷ trọng ảnh hưởng của từng đặc trưng mạng không chỉ giúp giải thích hiện tượng thắt cổ chai (bottleneck) mà còn là cơ sở để xây dựng các mô hình dự đoán hiệu năng mạng chính xác hơn khi truy cập dịch vụ Web.

1.2. Mục tiêu, câu hỏi nghiên cứu, phạm vi và phương pháp tổng quát

1.2.1. Mục tiêu nghiên cứu

Nghiên cứu hướng đến việc đo đạc và thu thập dữ liệu thực nghiệm từ 15 website công cộng phổ biến tại Việt Nam (Google, X, Facebook, ...) tại nhiều thời điểm khác nhau trong ngày, từ đó xác định mức độ tác động của các đặc trưng mạng, bao gồm thời gian phân giải DNS, thời gian thiết lập kết nối TCP, thời gian bắt tay TLS, độ trễ vòng RTT, kích thước phản hồi, mã trạng thái HTTP, thời điểm truy cập và loại hình domain, đến biến mục tiêu là thời gian phản hồi (response_time_ms). Trên cơ sở đó, báo cáo xây dựng các mô hình dự đoán từ cơ bản đến nâng cao nhằm ước lượng thời gian phản hồi dịch vụ Web dựa trên các thông số mạng đã thu thập được.

1.2.2. Câu hỏi nghiên cứu

Báo cáo đặt ra ba câu hỏi trọng tâm:

- Thứ nhất, các yếu tố đặc trưng mạng (DNS, TCP, TLS, RTT) ảnh hưởng đến tổng thời gian phản hồi ra sao?
- Thứ hai, các yếu tố ngữ cảnh như thời điểm truy cập, phân loại domain hay kích thước phản hồi ảnh hưởng như thế nào đến sự thời gian phản hồi của máy chủ?
- Thứ ba, độ phù hợp của các mô hình tuyến tính và phi tuyến tính đối với bài toán này ra sao? Liệu có tạo ra kết quả chính xác hơn Baseline không? Loại mô hình nào phù hợp nhất?

1.2.3. Phạm vi nghiên cứu

Nghiên cứu được giới hạn ở góc độ đo đạc phía người dùng đối với các dịch vụ Web công cộng, thông qua các giao thức mạng tiêu chuẩn DNS, TCP, TLS và HTTP/HTTPS.

Tập dữ liệu gồm 863 bản ghi thực tế (sau làm sạch), thu thập từ 15 tên miền hợp lệ, thuộc 8 nhóm dịch vụ phổ biến. Các bản ghi đồng thời tuân thủ nguyên tắc an toàn mạng bằng cách không thực hiện truy vấn với tần suất cao gây quá tải cho hệ thống máy chủ mục tiêu.

1.2.4. Phương pháp tổng quát

Quá trình nghiên cứu được triển khai qua ba giai đoạn chính.

Ở giai đoạn thu thập dữ liệu, một script Python được sử dụng để chủ động khởi tạo các truy vấn và ghi nhận chính xác thời gian hoàn thành của từng giai đoạn kết nối mạng.

Tiếp theo, dữ liệu được làm sạch, xử lý các giá trị khuyết thiếu và ngoại lệ, mã hóa các biến định tính, đồng thời phân tích mối tương quan giữa các biến và thời gian phản hồi.

Cuối cùng, ở giai đoạn mô hình hóa, mô hình Linear Regression được huấn luyện và so sánh với các mô hình Decision Tree và Random Forest Regressor trên tập Train/Test, có đối chứng với mô hình Baseline (Mean/Median Target).

1.3. Kết quả dự kiến

Nghiên cứu kỳ vọng đạt được một bộ dữ liệu đo đạc thực tế hoàn chỉnh, phản ánh chi tiết các thông số thời gian trong chồng giao thức Web, cùng với báo cáo phân tích tương quan định lượng làm rõ mối liên hệ giữa các thành phần độ trễ hạ tầng (DNS, TCP, TLS, RTT), các đặc trưng ngữ cảnh và tổng thời gian phản hồi.

Mô hình dự đoán thời gian phản hồi được xây dựng cần đạt độ chính xác cao, thể hiện qua các chỉ số MAE, RMSE và hệ số xác định R^2 vượt trội so với Baseline. Cuối cùng, báo cáo sẽ đi sâu đánh giá các trường hợp dự đoán có sai số lớn dưới góc nhìn Mạng máy tính để đưa ra một số lời lý giải cũng như rút ra hướng phát triển cho mô hình.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Các giao thức nền tảng của một phiên truy cập Web

2.1.1. DNS (Domain Name System)

DNS là dịch vụ ở tầng Ứng dụng, có nhiệm vụ ánh xạ tên miền dạng chữ dễ nhớ (ví dụ lms.ftu.edu.vn) thành địa chỉ IP mà máy tính có thể xử lý. DNS thường hoạt động trên UDP qua cổng 53 do kích thước gói tin nhỏ và cần tốc độ nhanh; tuy vậy vẫn có thể chuyển sang TCP khi cần truyền các phản hồi lớn hơn hoặc đồng bộ vùng dữ liệu.

2.1.2. TLS và HTTPS

TLS là giao thức bảo mật nằm trung gian giữa tầng Ứng dụng và tầng Vận chuyển, có nhiệm vụ thiết lập kênh truyền an toàn thông qua xác thực (authenticate) và mã hóa (encrypt) dữ liệu trao đổi.

HTTPS là việc chạy HTTP bên trong kênh truyền TLS, mặc định trên cổng TCP 443.

2.1.3. HTTP (Hypertext Transfer Protocol)

HTTP là giao thức trao đổi dữ liệu trụ cột của Web ở tầng Ứng dụng, hoạt động theo mô hình Request/Response. Client gửi HTTP Request chứa phương thức (GET, POST...) và metadata (headers, cookies), Server xử lý và trả về mã trạng thái (Status Code, ví dụ 200 OK, 404 NOT FOUND) cùng nội dung dữ liệu.

2.1.4. TCP (Transmission Control Protocol)

TCP thuộc tầng Vận chuyển, là giao thức hướng kết nối (connection-oriented) cung cấp dòng byte tin cậy, có thứ tự. Để đảm bảo điều này, TCP dùng bắt tay 3 bước (Three-way handshake) để thiết lập kết nối. Các cơ chế cốt lõi gồm: đánh số thứ tự byte, xác nhận tích lũy (ACK), truyền lại dữ liệu khi mất gói (retransmission), điều khiển luồng (flow control) và điều khiển tắc nghẽn (congestion control).

2.1.5. RTT (Round-Trip Time)

RTT là khoảng thời gian từ lúc một gói tin yêu cầu được gửi đi đến khi nhận được phản hồi tương ứng. RTT tại một thời điểm là tổng hợp nhiều thành phần độ trễ tại các nút mạng: độ trễ xử lý (processing delay), độ trễ hàng đợi (queueing delay), độ trễ truyền dẫn (transmission delay) và độ trễ lan truyền tín hiệu vật lý (propagation delay).

2.2. Throughput vs Goodput

Thông lượng (Throughput) là tốc độ truyền dữ liệu thực tế quan sát được qua một liên kết trong một khoảng thời gian nhất định, tính toàn bộ bit đẩy lên đường truyền, bao gồm cả dữ liệu ứng dụng, các thông tin điều khiển dư thừa (protocol headers) của các tầng, và các gói tin phải truyền lại (retransmissions) do lỗi.

Thông lượng hữu ích (Goodput) là tốc độ truyền dữ liệu hữu ích thực sự đến đích, loại bỏ hoàn toàn overhead (IP header, TCP/UDP header, Ethernet header, FCS) và dữ liệu truyền lại. Do đó luôn có quan hệ: $\text{Goodput} \leq \text{Throughput} \leq \text{Bandwidth}$.

- **Ảnh hưởng của kích thước Payload (Overhead tiêu đề):** UDP header chỉ tốn 8 byte, còn TCP header tiêu chuẩn tối thiểu 20 byte. Với cùng lượng overhead 58 byte, payload 100 byte chỉ đạt hiệu suất hữu ích khoảng 63,29%, trong khi payload 1200 byte đạt tới 95,39%. Payload lớn giúp Goodput tiệm cận Throughput.
- **Giới hạn tốc độ do RTT và cửa sổ (Window Size):** trong điều kiện lý tưởng không mất gói, thông lượng TCP bị giới hạn bởi $\text{Throughput} \leq \text{Window Size} / \text{RTT}$. Khi RTT lớn, để tận dụng tối đa băng thông danh nghĩa, cửa sổ gửi phải đủ lớn để đạt mức tích băng thông - độ trễ ($\text{BDP} = \text{Bandwidth} \times \text{RTT}$).
- **Ảnh hưởng của việc truyền lại (Retransmission):** Khi xảy ra mất gói, TCP tự động truyền lại các phân đoạn bị thiếu. Lượng dữ liệu truyền lại

này làm tăng Throughput trên liên kết vật lý nhưng bị khấu trừ hoàn toàn khỏi Goodput của ứng dụng.

2.3. Mất gói (Packet Loss) và cơ chế phản ứng của TCP

Trong mạng có dây truyền thống, mất gói chủ yếu xảy ra do tràn bộ đệm hàng đợi (queue/buffer overflow) tại các router trung gian khi tốc độ gói tin đến vượt quá khả năng phục vụ của liên kết đầu ra. TCP mặc định coi mọi trường hợp mất gói là dấu hiệu tắc nghẽn mạng; khi phát hiện mất gói (qua timeout hoặc duplicate ACK), TCP giảm ngay cửa sổ tắc nghẽn (cwnd) và chuyển sang chế độ truyền thận trọng để bảo vệ tài nguyên mạng.

Cụ thể hơn về động lực cwnd: ở pha Slow Start, cwnd bắt đầu từ 1 MSS (Maximum Segment Size) và tăng gấp đôi sau mỗi RTT nhận đủ ACK (tăng theo cấp số nhân) cho đến khi chạm ngưỡng ssthresh, sau đó chuyển sang Congestion Avoidance với tốc độ tăng tuyến tính (cộng thêm ~ 1 MSS mỗi RTT). Khi phát hiện mất gói qua 3 duplicate ACK, TCP Reno/NewReno giảm ngay cwnd còn một nửa (Fast Recovery) thay vì reset về 1 như timeout. Đây chính là cơ chế phi tuyến mà mô hình Linear Regression của đề tài không thể học được từ một hệ số góc cố định.

2.4. Jitter và cơ chế truy cập kênh Wi-Fi

Jitter là sự biến động về độ trễ giữa các gói tin khác nhau trong cùng một luồng dữ liệu, ảnh hưởng nghiêm trọng tới ứng dụng thời gian thực (VoIP, video streaming).

- Mạng Wi-Fi dùng cơ chế truy cập kênh CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) để tránh va chạm.
- Trước khi gửi, thiết bị phải lắng nghe kênh (carrier sensing) và, nếu bận, thực hiện thuật toán lùi thời gian ngẫu nhiên (backoff). Khi nhiều thiết bị cạnh tranh, độ trễ này biến thiên liên tục và tạo ra jitter lớn.

Bộ đếm backoff được chọn ngẫu nhiên trong cửa sổ tranh chấp (Contention Window), khởi đầu CWmin thường là 15 hoặc 31 khe thời gian (time slot, mỗi

slot cỡ 9–20 μs tùy chuẩn 802.11); mỗi lần va chạm hoặc không nhận ACK, CW tăng gấp đôi tới CW_{max} — cơ chế Binary Exponential Backoff này khiến độ trễ truy cập kênh không ổn định và cộng dồn vào thời gian phản hồi mà tập dữ liệu của đề tài không tách riêng được, vì phép đo chỉ nằm ở tầng Ứng dụng/Vận chuyển, không quan sát tầng Liên kết dữ liệu.

2.5. Phân rã Response Time

Response Time (thời gian từ lúc người dùng gửi yêu cầu đến khi nhận được phản hồi hoàn toàn) là tổng hợp các thành phần trễ lũy kế từ tầng Vật lý lên tầng Ứng dụng:

- **Độ trễ truyền dẫn (Transmission Delay):** thời gian đẩy toàn bộ bit của gói tin lên đường truyền vật lý, với L là kích thước gói tin và R là tốc độ liên kết.
- **Độ trễ lan truyền (Propagation Delay):** thời gian tín hiệu di chuyển qua khoảng cách D với vận tốc lan truyền v ($\sim 2 \times 10^8$ m/s). Tăng băng thông không giúp giảm độ trễ lan truyền.
- **Độ trễ hàng đợi (Queueing Delay):** thời gian gói tin chờ trong bộ đệm switch/router/AP trước khi được xử lý.
- **Độ trễ xử lý (Processing Delay):** thời gian router đọc header, giảm TTL, tính lại checksum và ra quyết định chuyển tiếp.
- **Độ trễ bắt tay giao thức:** trước khi trao đổi dữ liệu ứng dụng, hệ thống cần phân giải DNS (~ 1 RTT), bắt tay TCP 3 bước (~ 1 RTT) và bắt tay TLS ($\sim 1-2$ RTT). Nếu RTT mạng lớn, các bước này tiêu tốn 3–4 RTT trước khi byte HTTP hữu ích đầu tiên được gửi đi.

CHƯƠNG 3: DATASET VÀ PHƯƠNG PHÁP

3.1. Nguồn dữ liệu và cách thu thập

3.1.1. Nguồn dữ liệu

Nhóm đã thực hiện thu thập dữ liệu từ đa dạng các trang web phổ biến tại Việt Nam để có nguồn dữ liệu nhất quán, cũng như để có thể gửi nhiều yêu cầu và thực hiện phép đo mà không ảnh hưởng đến web. Ở dưới là 15 trang web được lựa chọn cho dự án:

1	https://x.com/
2	https://www.facebook.com/
3	https://www.google.com/
4	https://www.tiktok.com/
5	https://www.youtube.com/
6	https://shopee.vn/
7	https://tuoitre.vn/
8	https://www.wikipedia.org/
9	https://vietjack.com/
10	https://baomoi.com/
11	https://www.apple.com/
12	https://www.pinterest.com/
13	https://www.microsoft.com/
14	https://coccoc.com/
15	https://www.lazada.vn/

Bảng 3.1.1 – 15 URL phổ biến dùng để tạo dataset

3.1.2 Quy trình thu thập dữ liệu

Đầu tiên, code sẽ đo round-trip time bằng cách thực hiện lệnh ping, điều này được làm ngay trước khi gửi request để có thông số khớp nhất với rtt của khoảng khắc gửi request.

Việc gửi request và ghi lại các thông số cần thiết cho học máy được thực hiện bằng cách sử dụng thư viện PycURL, giao diện Python cho libcurl, một thư viện truyền tải tập tin hỗ trợ nhiều giao thức. Code sẽ thực hiện gửi GET request đến URL chỉ định, và viết lại phản hồi của server vào một biến. PycURL ghi lại một số tham số quan trọng trong quá trình trao đổi, những tham số liên quan đến dự án là:

- **NAMELOOKUP_TIME**: thời gian từ đầu -> tìm được địa chỉ IP cho tên miền.
- **CONNECT_TIME**: thời gian từ đầu -> hoàn thành TCP 3-way handshake.
- **APPCONNECT_TIME**: thời gian từ đầu -> hoàn thành TLS handshake.
- **TOTAL_TIME**: thời gian từ đầu -> người dùng nhận được phản hồi đầy đủ của server
- **HTTP_CODE**: mã phản hồi http nhận từ server

Cuối cùng, code sẽ decode phản hồi từ server và đo kích cỡ của nội dung phản hồi theo byte.

Code thu thập dữ liệu được chạy trong 8 thời điểm trong ngày với mỗi lần đo cách nhau ~3 tiếng, bắt đầu từ 00:00 nửa đêm. Mỗi lần chạy, code sẽ thực hiện 10 phép đo cho từng website. Điều này sẽ cho thấy toàn cảnh trải nghiệm dịch vụ web trong mọi khung giờ cũng như cung cấp thông tin cho việc huấn luyện mô hình học máy.

3.2 Mô tả dữ liệu

Variable	Kiểu dữ liệu	Đơn vị	Ý nghĩa	Vai trò
dns_time_ms	Numerical	ms	Thời gian chuyển đổi tên miền thành địa chỉ IP	Feature
tcp_connect_ms	Numerical	ms	Thời gian thuần của bắt tay TCP	Feature
tls_time_ms	Numerical	ms	Thời gian riêng của bắt tay TLS	Feature
rtt_ms	Numerical	ms	Round-trip time, thời gian khứ hồi khi gửi yêu cầu/tín hiệu đến khi nhận được phản hồi.	Feature
response_size	Numerical	byte	Kích thước response body	Feature
http_status	Categorical (int)	-	Mã trạng thái HTTP cuối cùng (luôn = 200 trong dữ liệu này)	Lọc dữ liệu
time_of_day	Numerical (HH-MM-SS)	-	Thời điểm bắt đầu request	Feature
domain_category	Categorical	-	Nhóm dịch vụ của tên miền, gán thủ công	Feature

response_time_ms	Numerical	ms	Tổng thời gian phản hồi của request	Target
------------------	-----------	----	-------------------------------------	--------

Bảng 3.2. Data dictionary

Dataset chứa thông tin về request được gửi tới 1 số web server và phản hồi nhận được, đo ở nhiều thời điểm khác nhau. Dữ liệu bao gồm các thông số biểu thị cho đặc trưng mạng, như thời gian khứ hồi, thời gian thiết lập kết nối với server và nhận lại phản hồi; cũng như thông tin về phản hồi nhận được, như mã http và kích cỡ của phản hồi; cùng với loại website và giờ/phút/giây request được gửi.

3.3. Xử lý dữ liệu

3.3.1. Tạo dataset

Dữ liệu thô cần được lưu vào một file .csv (Comma-separated values) để làm file input cho mô hình. Code thực hiện lưu thông tin thu được vào một dictionary với key và item lần lượt là string đại diện cho các feature và list đại diện cho các observation. Sau đó dictionary được biến đổi thành dataframe và được viết vào file .csv khởi tạo ở địa chỉ được chỉ định với thư viện pandas.

3.3.2 Xử lý dữ liệu

- **Missing value:** trường hợp không thể gửi request đến server nghĩa là:

1. Server bị sự cố hoặc không chấp nhận request
2. Vấn đề với kết nối mạng
3. Quy trình mở kết nối với server thất bại

Trong cả 3 trường hợp đều không thể hoàn thành quy trình trao đổi giữa người dùng và server, code báo lỗi. Nếu tạo exception, những observation này vẫn không thể đo được thời gian phản hồi, không có giá trị với dự án, code sẽ drop observation đó khỏi dataset.
-> Giá trị null không xảy ra.

- **Duplicate:** không xảy ra.

- **Kiểm tra tính nhất quán:** Loại bỏ các bản ghi có số đo vô lý:
 $\text{dns_time_ms} + \text{tcp_connect_ms} + \text{tls_time_ms} > \text{response_time_ms};$
 $\text{rtt_ms} > \text{tcp_connect_ms}.$
- **Chuẩn hóa categorical data:** Sử dụng phương pháp one-hot encoding
 Kết quả: 1200 $\rightarrow$ 863 bản ghi sau làm sạch cấu trúc.

3.3.3. Tách Train/Test và làm sạch thống kê

Tập dữ liệu sau khi làm sạch cấu trúc (863 bản ghi) được chia theo tỷ lệ 80/20 bằng hàm `train_test_split` (`random_state = 42`), thu được 690 bản ghi cho tập huấn luyện (train) và 173 bản ghi cho tập kiểm tra (test).

Outlier được phát hiện bằng thuật toán Isolation Forest (`contamination = 0.02`). Mô hình này được huấn luyện duy nhất trên tập train, sau đó áp dụng để lọc outlier trên cả hai tập train và test — không fit trên tập test, nhằm tránh rò rỉ dữ liệu (data leakage) và đảm bảo tính khách quan trong đánh giá mô hình. Kết quả:

<i>Tập dữ liệu</i>	<i>Trước lọc</i>	<i>Sau lọc</i>	<i>Số điểm loại bỏ</i>
<i>Train</i>	690	676	14
<i>Test</i>	173	172	1

Bảng 3.3.3 Tách tập dữ liệu Train/Test

CHƯƠNG 4: XÂY DỰNG CHƯƠNG TRÌNH VÀ THỰC NGHIỆM

4.1. Kiến trúc chương trình

Chương trình được tổ chức theo kiến trúc pipeline tuyến tính, trong đó mỗi giai đoạn xử lý được cài đặt thành một hàm độc lập nhằm đảm bảo tính tái lập và dễ kiểm thử:

Load → Clean → Analyze → Feature Engineering → Train → Evaluate → Visualize.

4.1.1. Thu thập và nạp dữ liệu

Tập dữ liệu được thu thập từ đa dạng các trang web phổ biến tại Việt Nam như được trình bày ở mục 3.1.1 và 3.1.2.

Nhóm lựa chọn lưu dataset dưới tên Dataset_finale.csv:

```
OUT_DIR = Path(__file__).resolve().parent.parent / "data"
OUT_DIR.mkdir(parents=True, exist_ok=True)
...
if Path(OUT_DIR / 'Dataset_finale.csv').exists():
    df.to_csv(OUT_DIR / 'Dataset_finale.csv', index=False,
header=False, mode='a')
else:
    df.to_csv(OUT_DIR / 'Dataset_finale.csv', index=False)
```

Đoạn mã 4.1.1. Lưu dữ liệu dưới tên Dataset_finale.csv

Hàm load_data() đọc tệp dữ liệu gốc data/Dataset_finale.csv và chuẩn hoá tên cột trước khi đưa vào các bước xử lý tiếp theo

4.1.2. Làm sạch dữ liệu

Việc xử lý dữ liệu trước khi chia tập train/test và phương pháp chia tập dữ liệu để chuẩn bị cho huấn luyện mô hình đã được trình bày ở mục 3.3.2 và 3.3.3.

Dưới đây là code để chia tập dữ liệu:

```
def clean(df, numeric_cols, test_size=0.2, random_state=42,
contamination=0.02):
    df = df.copy()
    ...
    train_df, test_df = train_test_split(df, test_size=test_size,
random_state=random_state)
    print(f"[split] train={train_df.shape}, test={test_df.shape}")
    ...
    return train_clean, test_clean
```

Đoạn mã 4.1.2.1. Chia tập train/test

Outlier được phát hiện bằng thuật toán Isolation Forest được huấn luyện ở trên tập train rồi áp dụng lên tập train và test như trình bày ở mục 3.3.3

```
model = IsolationForest(contamination=contamination,
random_state=random_state)
model.fit(train_df[numeric_cols])
train_pred = model.predict(train_df[numeric_cols])
test_pred = model.predict(test_df[numeric_cols])
train_clean = train_df[train_pred == 1].copy()
test_clean = test_df[test_pred == 1].copy()
return train_clean, test_clean
```

Đoạn mã 4.1.2.2. Xử lý outlier bằng thuật toán Isolation Forest

Việc chỉ fit mô hình trên tập train mô phỏng đúng tình huống triển khai thực tế, trong đó mô hình không được tiếp cận phân bố của dữ liệu kiểm tra trước khi đánh giá. Tham số `contamination = 0,02` được lựa chọn thủ công thay vì áp dụng ngưỡng z-score cố định, do Isolation Forest phù hợp hơn với phân bố lệch phải (right-skewed) đặc trưng của dữ liệu độ trễ mạng. Với tập dữ liệu hiện tại, bước này loại 14/690 bản ghi trên tập train và 1/173 bản ghi trên tập test.

4.1.3. Phân tích khám phá dữ liệu

Trước khi đưa dữ liệu vào huấn luyện, nhóm thực hiện thống kê mô tả trên tập train với hàm `analyze()` nhằm nắm được đặc trưng phân bố của từng biến số.

Với mỗi biến định lượng (`dns_time_ms`, `tcp_connect_ms`, `tls_time_ms`, `rtt_ms`, `response_size`, `response_time_ms`), các chỉ số được trích xuất gồm:

1. Số lượng quan sát (count)
2. Giá trị trung bình (mean)
3. Độ lệch chuẩn (std)
4. Giá trị nhỏ nhất/lớn nhất (min/max),
5. Các khoảng tứ phân vị (Q1 - 25%, Q2/median - 50%, Q3 - 75%).

Đây là bước cơ bản để phát hiện độ phân tán, độ lệch (skew) và dấu hiệu bất thường ban đầu trước khi áp dụng Isolation Forest ở bước làm sạch thống kê.

Các giá trị được lưu ở `results/tables/describe_train.csv`

- **Phân tích tương quan (Pearson):**

Sau bước feature engineering (mã hóa thời gian dạng cyclical, one-hot `domain_category`) (được trình bày ở mục 3.4 và 4.1.4), nhóm tính hệ số tương quan Pearson giữa từng đặc trưng số và biến mục tiêu `response_time_ms`:

Bước này chỉ thực hiện trên tập train, không đụng đến tập test. Chỉ số pearson giữa các feature và target là mối quan hệ tuyến tính vốn có giữa biến đầu vào và biến

mục tiêu, độc lập với bất kỳ mô hình cụ thể nào, không phải kết quả suy ra từ một model đã huấn luyện.

4.1.4. Trích xuất đặc trưng

Giai đoạn trích xuất đặc trưng được thực hiện bởi `feature_engineer()`, gồm ba phép biến đổi chính: mã hoá chu kỳ (cyclical encoding) cho biến thời gian, mã hoá one-hot cho biến phân loại `domain_category`, và chuẩn hóa dữ liệu:

- **Mã hoá chu kỳ thời gian (cyclical encoding):** Đặc trưng `time_of_day` (định dạng HH-MM-SS) được chuyển đổi thành hai biến `time_sin` và `time_cos` theo công thức:

$$time_sin = \sin(2\pi \times \frac{tổng\ giây}{86400})$$

$$time_cos = \cos(2\pi \times \frac{tổng\ giây}{86400})$$

Biến `time_of_day` ở dạng số nguyên (ví dụ 235959 và 000001) có khoảng cách số học rất lớn dù chỉ cách nhau hai giây trên thực tế; nếu đưa trực tiếp vào Linear Regression, mô hình sẽ học sai quan hệ tại ranh giới nửa đêm. Phép biến đổi sang tọa độ $(\sin \theta, \cos \theta)$ trên vòng tròn đơn vị bảo toàn tính tuần hoàn của thời gian trong ngày.

- **Mã hoá biến phân loại:**

Đặc trưng `domain_category` được mã hoá bằng phương pháp one-hot encoding (`drop_first=True`) trên 8 nhóm dịch vụ, thu được 7 cột nhị phân sau khi loại bỏ nhóm tham chiếu.

Danh mục các nhóm được fit trên tập train; tập test sau đó được `reindex` theo đúng cấu trúc cột của tập train, các danh mục không xuất hiện trong test được điền giá trị 0 (one-hot coding).

- **Chuẩn hoá dữ liệu:**

Việc chuẩn hoá dữ liệu là cần thiết vì các đặc trưng trong tập dữ liệu có đơn vị và biên độ rất khác nhau (ví dụ: `dns_time_ms`, `tcp_connect_ms` thường ở khoảng vài đến vài trăm mili-giây, trong khi `response_size` có thể lên tới hàng nghìn byte.). Nếu giữ nguyên thang đo gốc, các mô hình dựa trên khoảng cách hoặc tổ hợp tuyến tính như Linear Regression sẽ bị chi phối bởi các biến có giá trị tuyệt đối lớn (`response_size`), dẫn đến hệ số hồi quy bị lệch và khó so sánh mức độ ảnh hưởng thực sự giữa các đặc trưng.

Hàm `scale_features()` sử dụng `StandardScaler`, được fit chỉ trên tập train rồi áp dụng (transform) lên cả train và test, tránh rò rỉ thông tin vào quá trình đánh giá mô hình.

4.1.5. Huấn luyện mô hình

Nhóm lựa chọn bốn mô hình theo nguyên tắc tăng dần độ phức tạp, nhằm đánh giá xem mức độ phi tuyến và tương tác giữa các đặc trưng có thực sự cải thiện khả năng dự đoán `response_time_ms` hay không, thay vì chỉ chạy một model duy nhất mà không có cơ sở so sánh.

Mô hình	Vai trò
Mean	Đối chứng tối thiểu (Baseline)
Linear Regression	So sánh tuyến tính đơn giản
Decision Tree Regressor	So sánh phi tuyến đơn giản

Random Forest Regressor	So sánh ensemble phi tuyến
-------------------------	----------------------------

Bảng 4.1.5.1. Các mô hình và baseline được lựa chọn

Code của các mô hình:

```
def train_models(X_train, y_train):
    models = {}
    models['baseline_mean'] = MeanBaseline().fit(X_train, y_train)
    models['linear_regression'] = LinearRegression().fit(X_train,
y_train)
    models['decision_tree'] = DecisionTreeRegressor(
        max_depth=6, random_state=42).fit(X_train, y_train)
    models['random_forest'] = RandomForestRegressor(
        n_estimators=200, max_depth=8, random_state=42
    ).fit(X_train, y_train)
    return models
```

Đoạn mã 4.1.5.2 Huấn luyện bốn mô hình với cấu hình tái lập.

Cả bốn mô hình đều được huấn luyện trên cùng tập X_{train} , y_{train} (đã qua feature engineering và chuẩn hoá), đảm bảo tính công bằng khi so sánh ở phần *Đánh giá* (mục 4.1.6). Cấu hình siêu tham số cụ thể (max_depth , n_estimators , random_state ...) được trình bày riêng ở phần 4.2 Cấu hình thực nghiệm.

4.1.6. Đánh giá

Việc đánh giá mô hình được thực hiện độc lập trên tập test, tách biệt hoàn toàn khỏi dữ liệu huấn luyện, nhằm phản ánh đúng khả năng tổng quát hoá thay vì hiệu năng ghi nhớ trên dữ liệu đã thấy.

Nhóm sử dụng đồng thời bốn chỉ số hồi quy là MAE, RMSE, R^2 và MAPE (quy đổi thành $\text{Accuracy} = 100 - \text{MAPE}$), thay vì chỉ dùng một chỉ số duy nhất, do mỗi chỉ số phản ánh một khía cạnh sai số khác nhau:

- MAE: cho biết độ lệch trung bình theo đơn vị gốc (ms), dễ diễn giải trực quan
- RMSE: nhạy hơn với các sai số lớn (outlier), phù hợp để phát hiện trường hợp dự đoán lệch nghiêm trọng
- R^2 : đánh giá tỷ lệ phương sai của biến mục tiêu được mô hình giải thích, cho phép so sánh tương đối giữa các mô hình có thang đo khác nhau
- MAPE/Accuracy biểu diễn sai số dưới dạng phần trăm, hỗ trợ diễn giải mang tính thực tiễn hơn cho người đọc không chuyên về thống kê.

Việc kết hợp cả bốn chỉ số giúp tránh kết luận phiến diện nếu chỉ dựa vào một thước đo.

Cuối cùng, nhóm bổ sung bước phân tích các trường hợp dự đoán sai lệch lớn nhất (largest residuals), nhằm không dừng lại ở các chỉ số tổng hợp mà còn xác định liệu sai số có mang tính hệ thống (tập trung ở một số điều kiện mạng hoặc loại domain nhất định) hay chỉ là nhiễu ngẫu nhiên, làm cơ sở lý giải kết quả dưới góc độ mạng máy tính.

Kết quả định lượng cụ thể và phân tích chi tiết theo từng mô hình được trình bày ở Chương 5.

4.1.7. Trực quan hóa

Chương trình sinh tổng cộng 19 biểu đồ và 7 bảng kết quả dạng CSV:

Nhóm biểu đồ / bảng	Số lượng	Mục đích
Khám phá dữ liệu của đặc trưng, mục tiêu	1 bảng	Phân tích khám phá dữ liệu tập train.
Scatter đặc trưng - mục tiêu	6 biểu đồ	Quan sát trực tiếp quan hệ từng biến số với response_time_ms trên tập train.
Tương quan Pearson	1 bảng + 1 biểu đồ	Xếp hạng mức độ liên quan tuyến tính của từng đặc trưng với biến mục tiêu.
Hệ số / độ quan trọng đặc trưng	1 bảng	Đối chiếu mức ảnh hưởng từng đặc trưng giữa các mô hình

Kết quả đánh giá tổng hợp	1 bảng	MAE, RMSE, R^2 , Accuracy của bốn mô hình (Bảng 5.2)
Phân tích sai số của 3 mô hình học máy	3 bảng	Mười request có sai số dự đoán lớn nhất, tính riêng cho ba mô hình non-baseline
Chẩn đoán mô hình (actual_vs_predicted, residual_plot, residual_hist)	12 biểu đồ	Phân tích sai số của từng mô hình (3 biểu đồ $\times$ 4 mô hình)

Bảng 4.1.7. Tổng hợp các nhóm biểu đồ được sinh bởi chương trình.

4.2. Cấu hình thực nghiệm và khả năng tái lập

Toàn bộ thực nghiệm được thiết kế để đảm bảo khả năng tái lập và tránh rò rỉ dữ liệu. Mọi bước có yếu tố ngẫu nhiên như tách tập dữ liệu, Isolation Forest, Decision Tree, Random Forest đều sử dụng chung `random_state = 42`. Điều này cho phép tái lập chính xác 100% kết quả trên cùng dữ liệu đầu vào. Không có bước xử lý nào được fit trên tập test: việc loại bỏ outlier, mã hoá one-hot và chuẩn hoá đặc trưng (StandardScaler) đều được fit trên tập train rồi áp dụng sang tập test.

Cấu hình thực nghiệm của các mô hình:

Mô hình	Cấu hình
Baseline (Mean)	Dự đoán hằng số bằng trung bình response_time_ms trên tập train
Linear Regression	Cấu hình mặc định của scikit-learn
Decision Tree Regressor	max_depth = 6, random_state = 42
Random Forest Regressor	n_estimators = 200, max_depth = 8, random_state = 42

Bảng 4.2 Cấu hình Baseline và các mô hình.

Kết quả số (results/tables/model_results.csv) và toàn bộ biểu đồ (results/figures/) được sinh tự động khi chạy python main.py, không yêu cầu chỉnh sửa thủ công. Danh sách thư viện cần cài đặt và thứ tự chạy chương trình được liệt kê trong [README.md](#).

4.3. Hướng dẫn chạy lại thực nghiệm

Quy trình chạy lại thực nghiệm gồm sáu bước như sau:

1. Cài đặt môi trường bằng pip install -r requirements.txt (bao gồm pandas, numpy, matplotlib, scikit-learn, etc.).
2. Đặt Dataset_finale.csv vào thư mục data/ ở gốc project.
3. Chạy python src/main.py từ terminal, tại thư mục gốc project

4. Kiểm tra results/tables/model_results.csv để đối chiếu với bốn dòng số liệu (baseline, linear regression, decision tree, random forest) trong Bảng 5.2 của báo cáo.

5. Kiểm tra

- results/figures/feature_vs_target/ (6 biểu đồ)
- results/figures/pearson_correlations.png cùng results/tables/pearson_correlations.csv
- biểu đồ của từng mô hình tại results/figures/<tên_mô_hình>/ (3 biểu đồ $\times$ 4 mô hình, gồm actual vs predicted, residual plot, residual histogram) để đối chiếu với kết quả ở Chương 5.

6. Kiểm tra

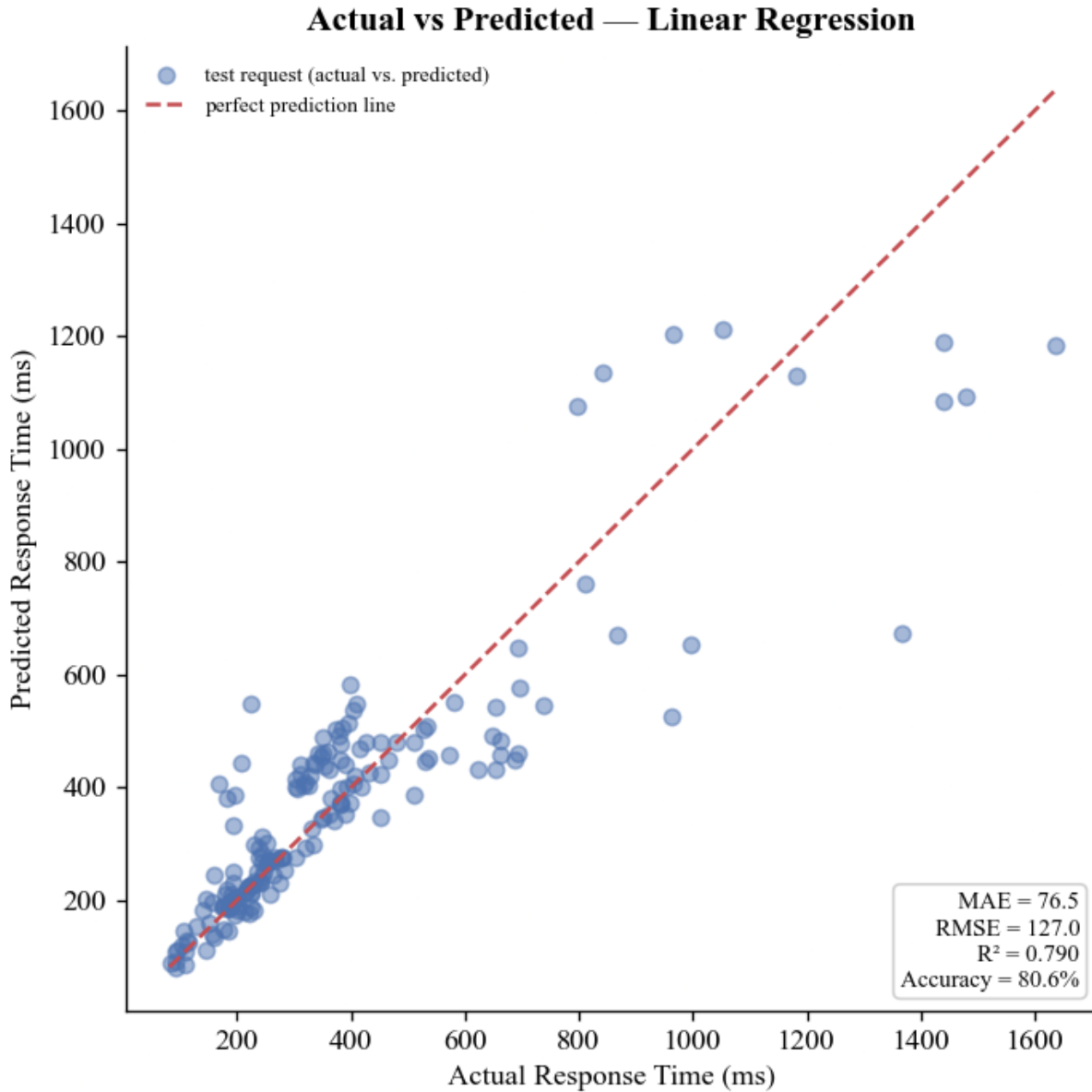
- results/tables/error_analysis_linear_regression_top10.csv
- results/tables/error_analysis_decision_tree_top10.csv
- results/tables/error_analysis_random_forest_top10.csv

Top 10 request sai lệch lớn nhất mỗi mô hình để đối chiếu với phân tích sai số ở mục 5.4.

CHƯƠNG 5: KẾT QUẢ VÀ THẢO LUẬN

5.1. Đánh giá và so sánh các mô hình học máy

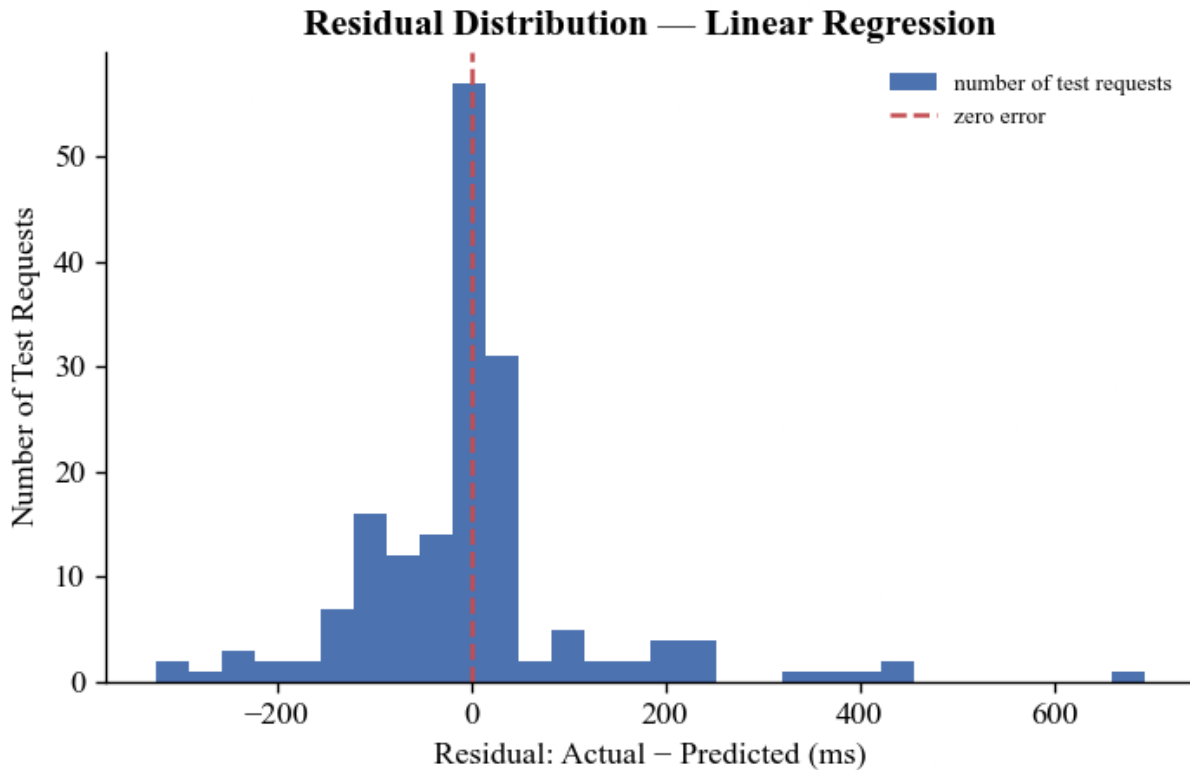
5.1.1 *Linear Regression*



Hình 5.1.1.1 Actual vs Predicted — Linear Regression

Với Linear Regression, phần lớn điểm bám sát đường dự đoán hoàn hảo ở vùng response_time_ms dưới 700 ms, tuy nhiên có dự đoán mang giá trị lớn hơn thực tế đáng

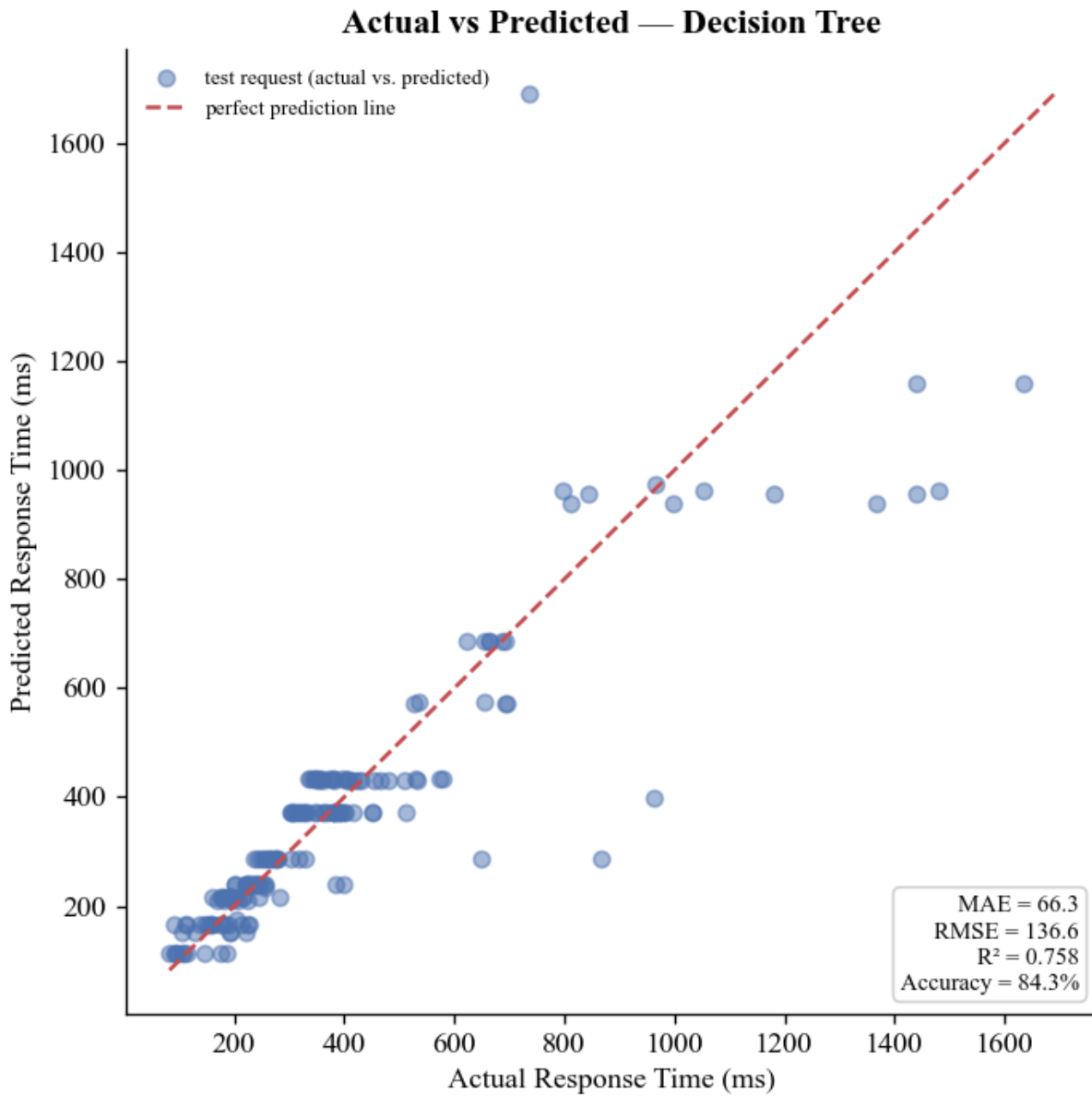
chú ý ở vùng 300-500ms; ở vùng cao hơn, mô hình có cả trường hợp dự đoán thấp hơn và cao hơn thực tế, tuy nhiên dự đoán thấp hơn chiếm phần lớn.



Hình 5.1.1.2. Phân bố residual (*Actual – Predicted*) — Linear Regression

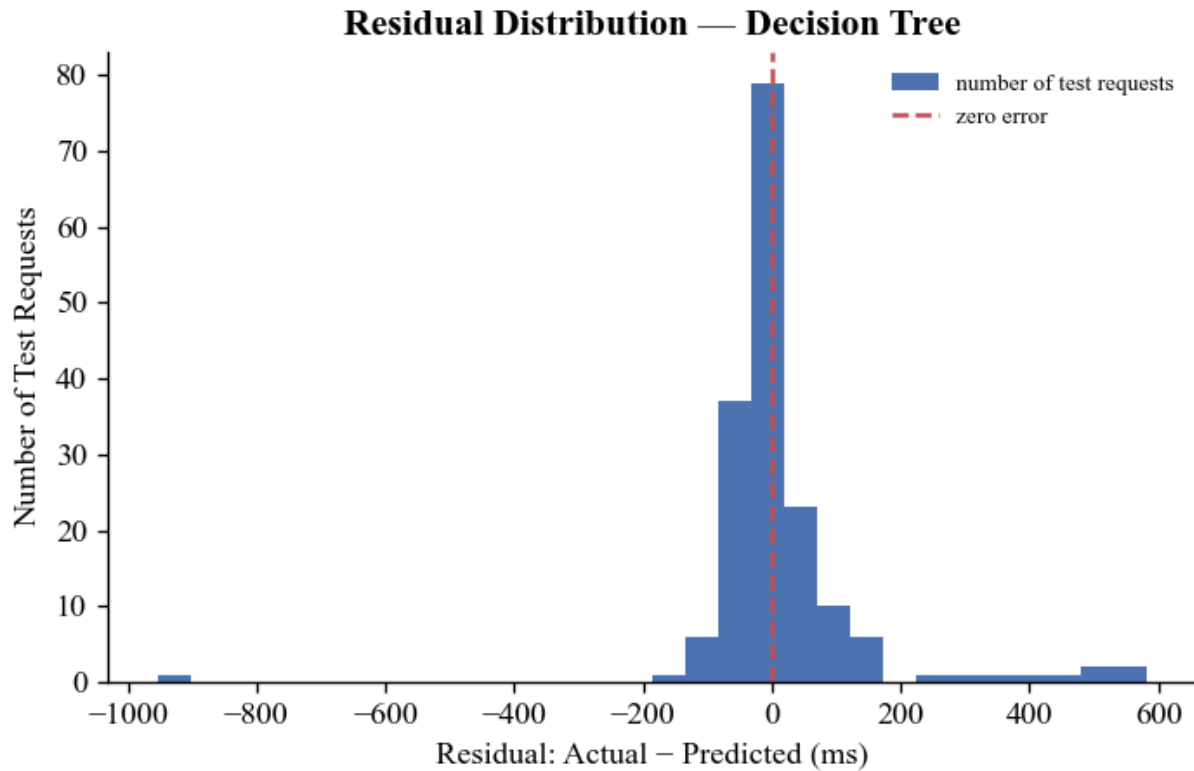
Phân bố residual của Linear Regression tập trung lớn ở 0. Phần lớn các sai số được thấy ở phạm vi -200 đến 100 ms. Một điểm đáng chú ý là các dự đoán với sai số cao nằm trong khoảng 300 đến 600 ms.

5.1.2 Decision Tree



Hình 5.1.2.1. Actual – Predicted — Decision Tree

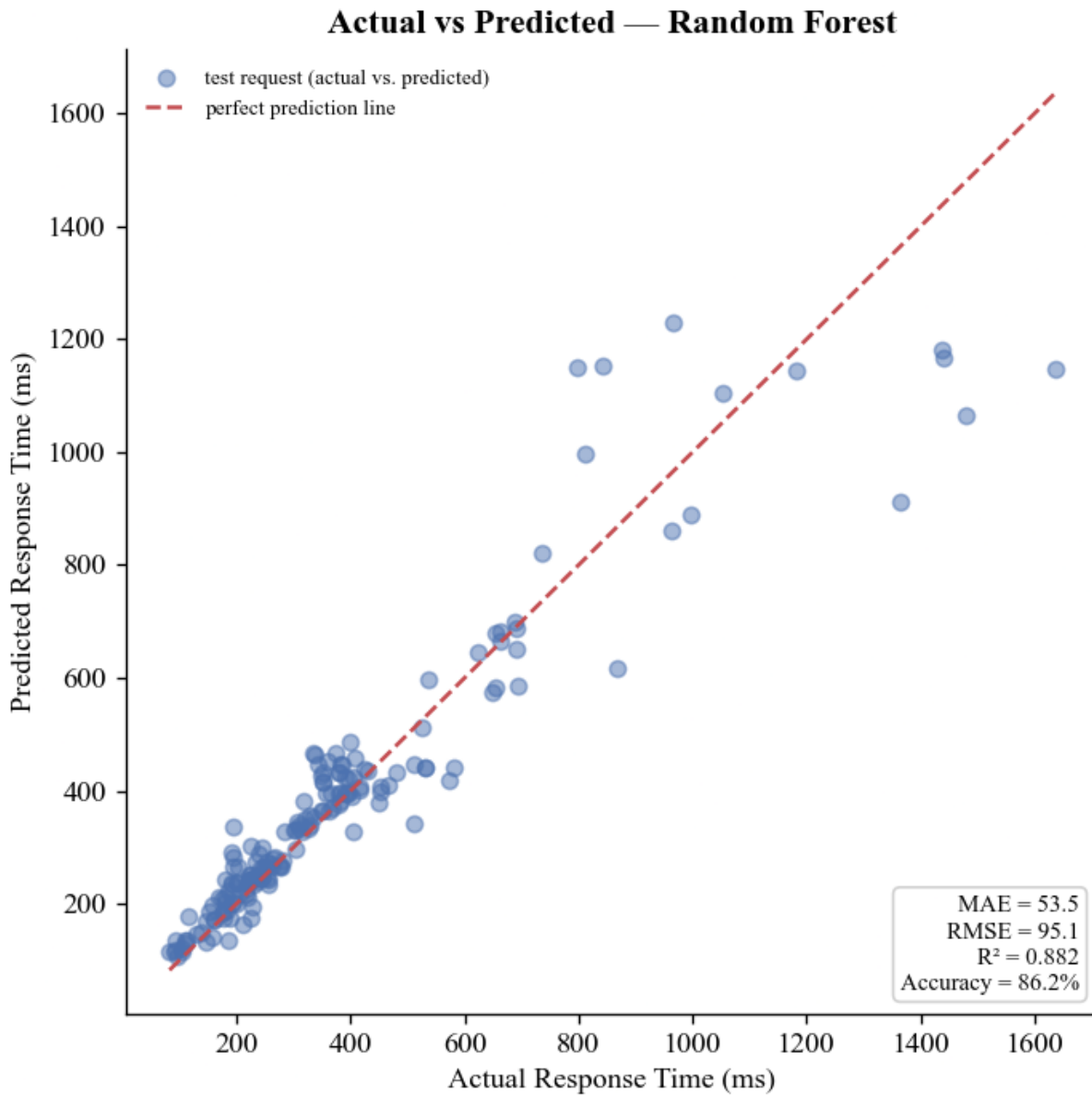
Với Decision Tree, có thể thấy các dự đoán ở phần response_time_ms đạt giá trị thấp hơn (600 ms đổ lại) có độ chính xác cao, với xu hướng dự đoán thấp hơn thực tế xảy ra khi thời gian phản hồi tăng lên.



Hình 5.1.2.2. Phân bố residual (Actual – Predicted) — Decision Tree

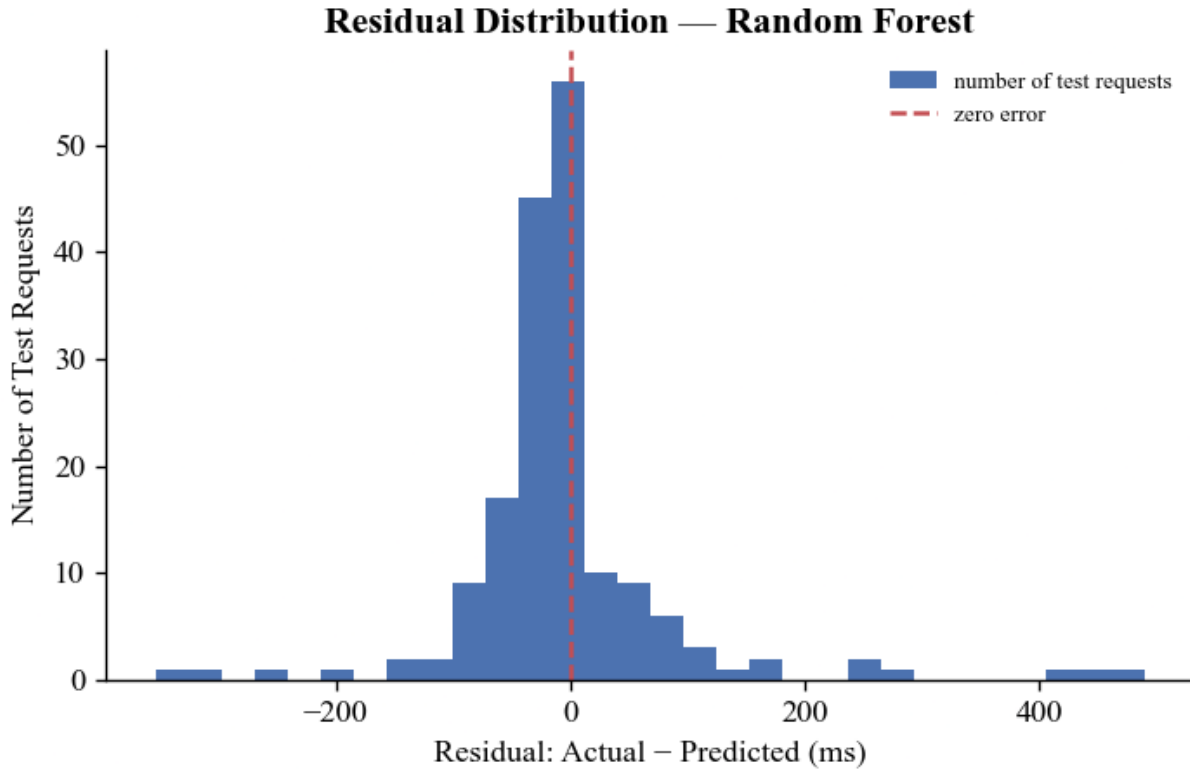
Phân bố residual có sự tập trung ở 0 ms. Nhìn chung, có thể thấy độ dư của mô hình này vượt trội so với Linear Regression, đáng chú ý là phân bố của các số dư trong phạm vi -200 đến 200 ms có sự cân bằng, thay vì xu hướng đoán cao hơn thực tế như Linear Regression; ở các giá trị cao hơn của biến target, mô hình có dự đoán dưới thực tế khá đáng kể (400 đến 600 ms), trừ một outlier ở gần -1000 ms.

5.1.3 Random Forest



Hình 5.1.3.1. Actual – Predicted — Random Forest

Với Random Forest, mức độ bám sát đường chéo tốt hơn rõ rệt ở vùng response_time_ms thấp và trung bình. Ở vùng giá trị response_time_ms cao, mô hình vẫn có xu hướng dự đoán thấp hơn thực tế tại một số điểm cực trị.



Hình 5.1.3.2. Phân bố residual (Actual – Predicted) — Random Forest

Phân bố residual của Random Forest tập trung quanh giá trị 0 với phần lớn sai số nằm trong khoảng ± 100 ms. Mô hình vẫn tồn tại một số điểm ngoại lai ở hai đuôi phân bố, tuy nhiên dao động gần với đường hoàn hảo hơn nhiều so với hai mô hình trước, số dư cao nhất chỉ ở khoảng 480 ms.

5.2. So sánh giữa các mô hình và với Baseline

Mô hình	MAE (ms)	RMSE (ms)	R ²	MAPE (%)	Accuracy (%)
Baseline (Mean)	187,1	277,5	-0.00008	66,0	33.96
Linear Regression	76,5	127,0	0,790	19,4	80.61

Decision Tree	66,3	136,6	0,758	15,7	84.27
Random Forest	53,5	95,1	0,882	13,8	86.12

Bảng 5.2. Kết quả đánh giá trên tập test (172 bản ghi)

Baseline (Mean) có MAE = 187,1 ms, RMSE = 277,5 ms và $R^2 \approx 0$ (-0,0001), phản ánh đúng bản chất của một mô hình không sử dụng thông tin đầu vào. Cả ba mô hình học máy đều cải thiện vượt trội so với baseline trên cả ba chỉ số.

Về MAE, Random Forest đạt sai số thấp nhất (53,5 ms), tiếp theo là Decision Tree (66,3 ms) và Linear Regression (76,5 ms) — tương ứng mức giảm lần lượt 71,4%, 64,6% và 59,1% so với baseline.

Về RMSE, Random Forest tiếp tục dẫn đầu (95,1 ms), sau đó là Linear Regression (127,0 ms) và Decision Tree (136,6 ms). Đáng chú ý, Decision Tree có MAE thấp hơn Linear Regression nhưng RMSE lại cao hơn, cho thấy mô hình này mắc một số sai số lớn cục bộ dù sai số trung bình nhỏ.

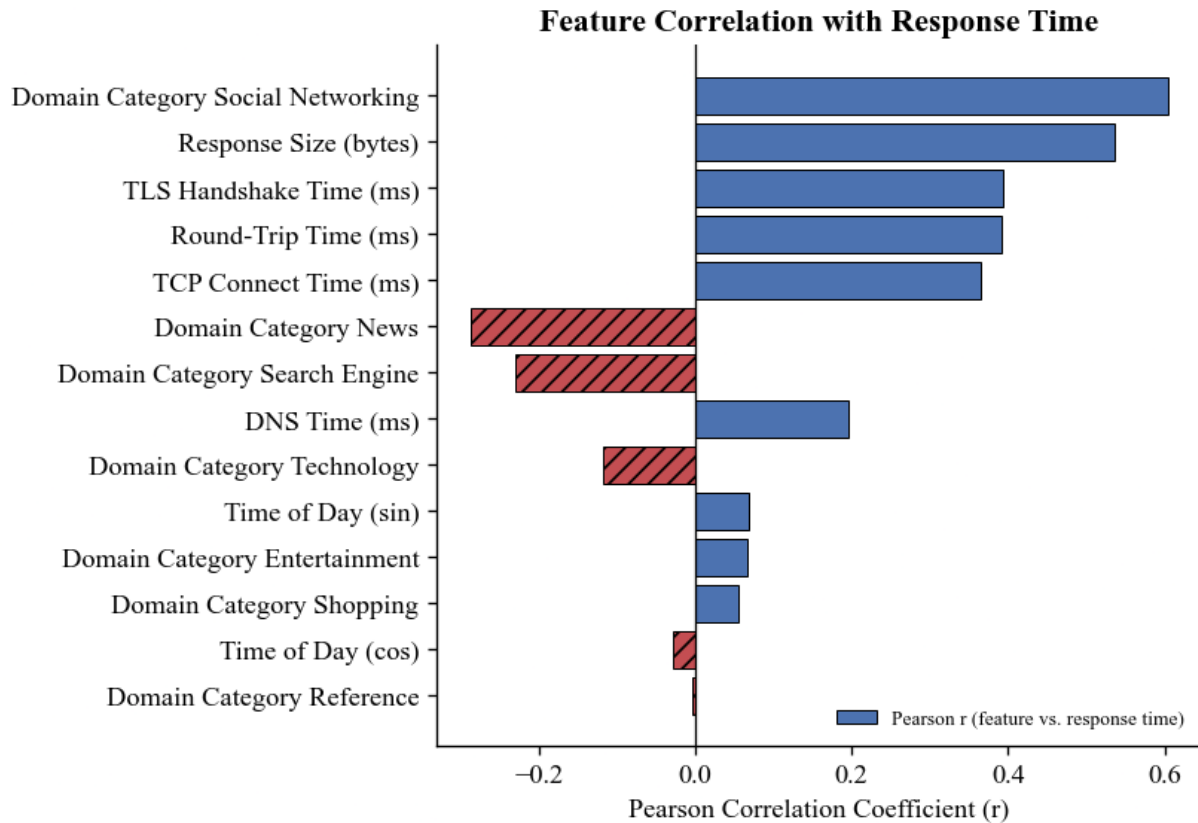
Về R^2 , Random Forest đạt giá trị cao nhất (0,882), giải thích được phần lớn phương sai của biến mục tiêu; Linear Regression đạt 0,790, cao hơn Decision Tree (0,758).

Tổng hợp cả ba chỉ số, Random Forest là mô hình tốt nhất, vượt trội cả về độ chính xác trung bình (MAE) lẫn mức độ ổn định trước sai số lớn (RMSE) và khả năng giải thích phương sai (R^2)

5.3. Phân tích feature

Kết quả trích xuất độ quan trọng đặc trưng (feature importance) từ mô hình Random Forest (results/tables/model_coefficients.csv) cho thấy response_size là đặc trưng có ảnh hưởng lớn nhất đến response_time_ms, với importance $\approx 0,637$.

Chỉ số này vượt trội so với `rtt_ms` đứng thứ hai ($\approx 0,175$). Tiếp theo là ba đặc trưng thời gian bắt tay `tls_time_ms`, `dns_time_ms` và `tcp_connect_ms` có mức đóng góp tương đương nhau ($0,031-0,037$). Các đặc trưng còn lại, bao gồm `time_sin`, `time_cos` và các nhóm dịch vụ khác, có ảnh hưởng không đáng kể.



Hình 5.3.1 Tương quan Pearson giữa từng đặc trưng (đã one-hot) và `response_time_ms` trên tập train (676 điểm)

Phân tích tương quan Pearson trên tập train (676 quan sát, sau one-hot encoding), thể hiện ở Hình 5.3.1, cho kết quả phần lớn nhất quán với phân tích độ quan trọng đặc trưng ở quy mô tổng thể. Biến `domain_category social networking` có tương quan dương mạnh nhất với `response_time_ms` ($r \approx 0,60$), theo sau là `response_size` ($r \approx 0,54$), `tls_time_ms` và `rtt_ms` ở mức gần bằng nhau ($r \approx 0,39$), và `tcp_connect_ms` ($r \approx 0,37$).

Ngược lại, hai nhóm dịch vụ News và Search engine có tương quan âm đáng kể ($r \approx -0,29$ và $-0,23$), cho thấy các request thuộc hai nhóm này có xu hướng đạt `response_time_ms` thấp hơn trung bình một cách hệ thống. Biến `time_of_day`, dưới cả hai dạng mã hoá sin và cos, gần như không tương quan với biến mục tiêu.

Đáng chú ý, `tls_time_ms` và `rtt_ms` có r Pearson gần như bằng nhau ($\approx 0,39$ mỗi biến) nhưng importance trong Random Forest lại chênh lệch lớn (`rtt_ms` $\approx 0,175$ so với `tls_time_ms` $\approx 0,031$). Điều này có thể giải thích từ chính cơ chế giao thức: như trình bày ở mục 2.5, bắt tay TLS tốn khoảng 1–2 vòng RTT, nghĩa là `tls_time_ms` về bản chất là một hàm phụ thuộc trực tiếp vào `rtt_ms` (cộng thêm phần xử lý mã hoá), chứ không phải một nguồn thông tin độc lập. Khi `rtt_ms` đã được đưa vào mô hình, phần biến động của `tls_time_ms` phần lớn chỉ lặp lại tín hiệu mà `rtt_ms` đã mang, nên Random Forest không cần tách riêng đóng góp của nó, toàn bộ "ngân sách" chia nhánh dồn vào `rtt_ms`, khiến `tls_time_ms` bị đánh giá thấp hơn nhiều so với mức tương quan tuyến tính đơn lẻ của nó.

Kết quả này phù hợp với cơ sở lý thuyết trình bày ở mục 2.2 và 2.5: theo công thức độ trễ truyền dẫn $\text{transmission delay} = L/R$, kích thước phản hồi (L) càng lớn thì thời gian đẩy dữ liệu lên đường truyền càng dài; đồng thời, với giao thức TCP, các phản hồi có kích thước lớn còn phải trải qua nhiều chu kỳ RTT trong pha Slow Start (mục 2.3) trước khi đạt tốc độ truyền tối đa. Điều này giải thích độ tương quan cao giữa biến `response_size` và `response_time_ms`, đồng thời giải thích độ tương quan của các biến `domain_category` với `response_time_ms`: những nhóm `domain_category` chứa website với kích cỡ phản hồi lớn sẽ có thời gian phản hồi lớn tương đương.

5.4. Các trường hợp dự đoán sai

5.4.1. Trường hợp model dự đoán thời gian nhanh hơn thực tế (Underprediction)

- **Model không nhìn thấy Packet Loss:** Trong trường hợp một packet chuyển chậm hoặc bị ngắt trên đường trung chuyển, TCP sẽ kích hoạt cơ chế Retransmission Timeout (RTO) hoặc Fast Retransmit - tuy nhiên model sẽ chỉ dự đoán thời gian phản hồi dựa trên giá trị RTT và DNS thông thường mà bỏ qua hoàn toàn khoảng delay từ 200 đến 1000 ms từ chính việc chuyển/gửi lại packet
- **Độ trễ từ chính server (Server-Side Delay):** Một request có thể tốn ít thời gian handshake và payload khiến cho model dự đoán thời gian phản hồi thành rất nhanh; tuy nhiên nếu chính server gốc phải chịu độ trễ từ một nguyên nhân khác (vd: xử lý các request trước đó, gửi lại một packet từ cơ chế Fast Retransmit, ...) thời gian phản hồi thực tế sẽ lâu hơn thời gian được dự đoán đáng kể
- **TCP khởi động chậm khi xử lý file lớn:** Với những file nhiều megabyte, congestion window của TCP sẽ tăng theo cấp bội (exponential) thông qua cơ chế Slow Start thay vì theo cấp tuyến tính (Linear), khiến cho model đánh giá thấp tốc độ chuyển file và phản hồi

Dữ liệu cho trường hợp này có thể được nhìn thấy ở hàng 3 của bảng `error_analysis_random_forest`. Ở đây, Handshake Overhead = 88.58ms, Transfer Time = 1,277.09ms và Effective Throughput = 0.31 MB/s.

Vì thời gian handshake lẫn RTT đều rất thấp, lần lượt ở mức 88ms và 26.59ms nên có thể loại trừ hai nguyên nhân là DNS/TCP/TLS delay và khoảng cách vật lý cao. Và chính vì thời gian transfer lên đến hơn 1.27 giây dù kích thước gói file rất thấp (403 KB), hai nguyên nhân chính có thể được chỉ ra là hiện tượng mất packet giữa dòng chuyển (mid-stream TCP packet loss dẫn đến Retransmission Timeout) và độ trễ từ chính phía server

5.4.2. Trường hợp model dự đoán thời gian phản hồi chậm hơn thực tế (Overprediction)

- **Tái sử dụng kết nối HTTP (Keep-Alive/ HTTP/2 & HTTP/3):** Nếu trình duyệt tái sử dụng một kết nối TCP/TLS mở, độ trễ đến từ setup overhead là rất nhỏ (gần như 0ms) khiến cho model đánh giá quá cao thời gian phản hồi
- **Trường hợp chèn nhớ đệm đặc biệt ở CDN (Edge CDN Caching):** Model đôi khi có thể đánh giá độ trễ cao do những lần delay từ server (server-side delays) trước đó. Nếu một request được cache ở gần một Point of Presence (PoP) như Cloudflare hoặc Akamai, thời gian trung chuyển sẽ bỏ qua thời gian xử lý của máy chủ gốc hoặc các route trung chuyển dài và trở nên nhanh hơn đáng kể

Dữ liệu cho trường hợp này có thể được nhìn thấy ở hàng số 8 của bảng `error_analysis_random_forest`. Nếu so sánh với hàng số 5, có thể thấy rằng dù cả 2 request đều xảy ra gần như cùng lúc tại cùng một khung giờ ngoài cao điểm, thời gian phản hồi của dữ liệu hàng 8 chỉ lâu hơn một chút dù khối lượng payload cao hơn đáng kể (1.32 MB so với 1.12 MB của hàng số 5). Với khối lượng payload 1.32 MB tương đối lớn cần chuyển, giao thức TCP Congestion Window khi đạt trạng thái mở rộng tối đa sẽ cho phép dữ liệu được chuyển đi ở dung lượng băng thông cao nhất, giúp giảm thiểu thiệt hại thời gian mà độ trễ từ handshake overhead mang lại.

Và vì các mô hình cây (tree model) và tuyến tính (linear regression) thường hay đánh giá cao độ trễ biên với mỗi kilobyte thêm vào (nếu mô hình dùng nguyên tắc đánh giá tuyến tính), có thể chỉ ra rằng trong trường hợp này, mô hình đã bỏ qua cơ chế điều chỉnh payload dưới tuyến tính (sub-linear payload scaling) đã được nhắc đến ở trên

5.5. Giới hạn của mô hình

- **Tính phi tuyến tính của thuật toán TCP Slow Start:** Việc dự đoán thời gian phản hồi bằng Linear Scaling đang bỏ qua cơ chế kiểm soát tắc nghẽn (congestion control) của TCP. Thông thường, bên gửi sẽ bắt đầu với Congestion Window tương đối nhỏ và liên tục nhân đôi sau mỗi RTT đếm được; và các file lớn sẽ đi qua đường cong thông lượng theo cấp bội (exponential throughput curve) trước khi đạt dung lượng băng thông tối đa, khiến cho thời gian phản hồi cao hơn nhiều so với thời gian mà Linear Regression dự đoán

- **Bỏ qua thời gian xử lý phía Server và cơ chế của CDN:**

domain_category là biến proxy tĩnh, không phân biệt được giữa một asset tĩnh được gửi từ hệ thống mạng Edge CDN (độ trễ server thấp, RTT gần bằng 0) và một request database động (dynamic data request) từ server gốc (độ trễ server cao, RTT cao)

- **Tính tương quan của domain_category:** tương quan cao nhóm Social networking cần được diễn giải thận trọng: các tên miền thuộc nhóm này trong tập dữ liệu (X, Facebook, TikTok, Pinterest) đều có đặc điểm trả về payload lớn — hình ảnh, mã kịch bản, nội dung động. Vì vậy, domain_category trong trường hợp này một phần đóng vai trò biến đại diện gián tiếp cho response_size, chứ không phản ánh một cơ chế độc lập. Đây là một giới hạn cần lưu ý khi diễn giải hệ số và độ quan trọng đặc trưng của mô hình.
- **Điểm gãy (Breaking Point):** Ở hệ thống giám sát theo thời gian thực hoặc môi trường có nhiều yếu tố xử lý cùng lúc cao (high concurrency), pipeline dễ dính hiện tượng tràn bộ đệm (buffer bloat) hoặc queue delay tại cái router trung chuyển - những thứ khiến cho biến RTT tĩnh khó đoán chính xác thời gian phản hồi vì queue delay chiếm phần lớn thời gian trễ.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.

6.1. Tóm tắt kết quả

Báo cáo đã nghiên cứu các kiến thức về quá trình trao đổi thông tin giữa người dùng và máy chủ, bao gồm các giao thức nền tảng của thiết lập kết nối giữa hai máy, các thành phần tạo ra và các nhân tố - thuộc nhóm đặc trưng mạng và ngữ cảnh - ảnh hưởng đến thời gian phản hồi của máy chủ. Nội dung báo cáo cho thấy:

- Mức độ ảnh hưởng của một số nhân tố đặc trưng mạng (thời gian thiết lập DNS, TCP, TLS, thời gian khử hồi) và ngữ cảnh (loại domain, kích cỡ và mã http phản hồi, thời gian gửi request) với thời gian phản hồi.
- Kết quả thử nghiệm dự đoán thời gian phản hồi khi biết các yếu tố trên sử dụng mô hình học máy tuyến tính: Linear Regression, và phi tuyến tính: Decision Tree, Random Forest. Từ đó đánh giá hiệu quả của từng mô hình.
- Diễn giải những kết quả nhận được dưới góc nhìn mạng máy tính.

Trên một dataset với 848 bản ghi (sau khi làm sạch dữ liệu và loại bỏ outlier bằng Isolation Forest), thu thập từ 15 website công khai phổ biến tại Việt Nam, nhóm đã tìm ra kết quả như sau:

- Phân tích độ quan trọng đặc trưng và tương quan Pearson đồng thuận chỉ ra yếu tố có ảnh hưởng lớn nhất đối với thời gian phản hồi là kích cỡ phản hồi, sau đó với cách biệt khá lớn là thời gian khứ hồi (round-trip time).
- Trong 3 mô hình học máy, Random Forest có hiệu quả vượt trội, với chỉ số MAE, RMSE, R^2 lần lượt là 53,5 ms, 95,1 ms và 0,882. Decision Tree có độ chính xác trung bình cao hơn so với Linear Regression, tuy nhiên có R^2 và RMSE tệ hơn, cho thấy sai số cục bộ cao mặc dù sai số trung bình thấp hơn. Cả 3 mô hình đều hiệu quả vượt trội so với Baseline, tính bằng trung bình của biến cần dự đoán (thời gian phản hồi).

6.2. Trả lời câu hỏi nghiên cứu

6.2.1. Các yếu tố đặc trưng mạng (DNS, TCP, TLS, RTT) ảnh hưởng đến tổng thời gian phản hồi ra sao?

Trong bốn đặc trưng này, `rtt_ms` có ảnh hưởng lớn nhất trong mô hình Random Forest (importance $\approx 0,175$), trong khi `dns_time_ms`, `tcp_connect_ms` và `tls_time_ms` có importance nhỏ và tương đương nhau (0,031–0,037).

Tuy nhiên, dưới góc độ tương quan tuyến tính đơn biến (Pearson), `tls_time_ms` và `rtt_ms` lại gần như bằng nhau ($r \approx 0,39$). Sự khác biệt này không mâu thuẫn mà phản ánh đúng bản chất giao thức: theo mục 2.5, bắt tay TLS vốn tiêu tốn 1–2 vòng RTT, nên `tls_time_ms` phần lớn là hệ quả phái sinh của `rtt_ms` chứ không phải một nguồn thông tin độc lập; khi `rtt_ms` đã có trong mô hình, phần đóng góp riêng của `tls_time_ms` bị thu hẹp lại do trùng lặp thông tin (đa cộng tuyến).

Nhìn chung, các bước bắt tay (DNS, TCP, TLS) chỉ chiếm tỷ trọng tương đối nhỏ trong tổng response time so với RTT và kích thước phản hồi.

6.2.2. Các yếu tố ngữ cảnh (thời điểm truy cập, phân loại domain, kích thước phản hồi) ảnh hưởng thế nào?

`Response_size` là yếu tố ảnh hưởng lớn nhất trong toàn bộ đặc trưng (importance $\approx 0,637$; $r \approx 0,54$), nhất quán với lý thuyết $\text{transmission delay} = L/R$. `domain_category`, đặc biệt nhóm Social networking ($r \approx 0,60$) và hai nhóm News, Search engine ($r \approx -0,29$ và $-0,23$), cũng có tương quan đáng kể.

Tuy nhiên mục 5.5 đã lưu ý tương quan này một phần phản ánh gián tiếp `response_size` của các domain trong từng nhóm, chứ không hoàn toàn là một cơ chế độc lập. Ngược lại, `time_of_day` (qua `time_sin`, `time_cos`) gần như không có ảnh hưởng đến thời gian phản hồi.

6.2.3. Mô hình tuyến tính và phi tuyến tính phù hợp ra sao? Có vượt Baseline không? Mô hình nào tốt nhất?

Cả ba mô hình học máy đều vượt Baseline rõ rệt (giảm MAE từ 59,1% đến 71,4%).

Trong đó, Random Forest cho kết quả tốt nhất trên cả ba chỉ số MAE, RMSE và R^2 . Linear Regression, do chỉ học được một hệ số góc cố định, không nắm bắt được các cơ chế phi tuyến như TCP Slow Start hay Congestion Avoidance (mục 2.3, 5.5), nên có sai số cao hơn Decision Tree và Random Forest. Điều này cho thấy các mối quan hệ phi tuyến tồn tại thực sự trong dữ liệu và có ý nghĩa với bài toán dự đoán thời gian phản hồi.

6.3. Hạn chế

6.3.1. Thiếu sót của mô hình

Mô hình Linear Regression không mô phỏng được các cơ chế phi tuyến của TCP (Slow Start, Congestion Avoidance), dẫn đến sai số hệ thống ở các trường hợp có độ trễ lớn.

Dữ liệu đo ở tầng Ứng dụng/Vận chuyển, không quan sát được tầng Liên kết dữ liệu, nên các nguồn jitter từ cơ chế truy cập kênh Wi-Fi (CSMA/CA, Binary Exponential Backoff) không được tách riêng và có thể lẫn vào nhiễu của mô hình.

Model chưa có phép đo để tính đến trường hợp có điều kiện tải cao (high concurrency) hoặc có hiện tượng buffer bloat/queueing delay tại router trung chuyển.

6.3.2. Hạn chế của mô hình

Domain_category là biến proxy tĩnh, độ tương quan quan sát được với response_time_ms có thể một phần là hệ quả gián tiếp của response_size chứ không phản ánh đúng bản chất nhóm dịch vụ.

Các đặc trưng bắt tay (tcp_connect_ms, tls_time_ms, rtt_ms) mang thông tin trùng lặp lẫn nhau do đều là các mốc thời gian tuần tự trên cùng một kết nối, khiến cho một số biến bị đánh giá thấp hơn mức tương quan thực tế của chúng với biến mục tiêu.

6.4. Hướng phát triển

Từ các giới hạn được diễn giải ra, nhóm đưa ra một số hướng phát triển:

- Bổ sung các đặc trưng phản ánh tình trạng kết nối (ví dụ: cờ đánh dấu request có phải truyền lại/Retransmission hay không, hoặc số lượng hop) để mô hình có thể tách bạch ảnh hưởng của packet loss khỏi độ trễ mạng thông thường.
- Thay thế domain_category bằng đặc trưng phản ánh trực tiếp cơ chế phục vụ (ví dụ: có/không có CDN, loại nội dung tĩnh/động) để giảm hiện tượng biến proxy gián tiếp cho response_size.
- Xem xét loại bỏ hoặc kết hợp (ví dụ: PCA) các đặc trưng bắt tay có tương quan cao với nhau (dns/tcp/tls/rtt) để giảm đa cộng tuyến, giúp importance phản ánh đúng hơn vai trò thực sự của từng thành phần giao thức.
- Thử nghiệm các mô hình phi tuyến mạnh hơn (Gradient Boosting, XGBoost) hoặc mô hình chuỗi thời gian nếu mở rộng số lượng thời điểm đo, nhằm khai thác tốt hơn các quan hệ phi tuyến đã quan sát được ở Random Forest.
- Mở rộng quy mô thu thập dữ liệu (nhiều domain hơn, nhiều thời điểm đo hơn trong ngày/tuần) để tăng độ tin cậy thống kê của các nhóm domain_category hiện có ít quan sát.

TÀI LIỆU THAM KHẢO

- [1] Ngô Hải Anh, Giáo trình Mạng máy tính: Nguyên lý, giao thức, an toàn và phân tích dữ liệu mạng
- [2] Google Data, Aggregated, anonymized Google Analytics data from a sample of mWeb sites opted into sharing benchmark data, n=3.7K, Global, March 2016, from <https://blog.google/products/admanager/the-need-for-mobile-speed/>
- [3] Rescorla, E. (2018). RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. IETF.
- [4] Fielding, R., Nottingham, M., & Reschke, J. (2022). RFC 9110: HTTP Semantics. IETF
- [5] Eddy, W. (2022). RFC 9293: Transmission Control Protocol (TCP). IETF
- [6] Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson.
- [7] Borman, D., Braden, R., Jacobson, V., & Scheffenegger, R. (2014). RFC 7323: TCP Extensions for High Performance. IETF
- [8] IEEE Standards Association. (2021). IEEE Std 802.11-2020: IEEE Standard for Information Technology [1] Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications. IEEE
- [9] PycURL –A Python Interface To The cURL library, from <https://pycurl.github.io/docs/latest/index.html>
- [10] IBM, What is a decision tree?, from www.ibm.com/think/topics/decision-trees
- [11] IBM, What is a decision tree?, from www.ibm.com/think/topics/linear-regression
- [12] IBM, What is a decision tree?, from www.ibm.com/think/topics/random-forest